

一、OpenEmbedded 与 Poky 的关系

OpenEmbedded (OE) 是一个通用的嵌入式Linux构建框架，提供了一套元数据 (recipes) 和构建工具 (如BitBake)，支持多种硬件架构和软件包管理方式。其核心组件 OpenEmbedded-Core (OE-Core) 是经过严格筛选的元数据集，包含约830个核心配方 (recipes)，适用于跨平台构建。

Poky是Yocto项目的参考构建系统，基于OE-Core和BitBake构建而成，属于OpenEmbedded的一个发行版，提供开箱即用的构建环境。

二、smart-build简介

smart-build下的meta-rt-smart可以看作是自定义的一个层 (Layer)，可以被添加到Poly之中。每个层下可以包含多个配方 (recipes)，配方是通过BB文件定义的。每个BB文件可以定义多个任务 (Task)。

因此在接入Poky系统之后，可以通过bitbake方式进行指定架构 (如aarch64, riscv64等) 工具链的安装，以及busybox(用于生成文件系统image), rt-smart的编译。

三、Layers创建注意事项

Layer及Recipes目录结构必须严格参考如下结构。

meta-myapp	#Layer定义必须以"meta-"为前缀
├─ recipes-example	#recipes定义必须以"recipes-"为前缀
│ └─ example	#这一层目录不可或缺，名称随意，bb文件不能直接放到这一层
│ └─ example_0.1.bb	#bb文件必须放到这一层，且必须带"_version"版本信息
├─ conf	#每个Layer必须包含conf/layer.conf配置文件
└─ layer.conf	

四、配置文件layer.conf说明

除了常规配置之外，额外增加了PATH环境变量的设置。

Bitbake运行的是一套独立的隔离环境，自定义的Task里面需要用到一些系统工具，所以需要将系统工具路径添加进来；后面的是Toolchain的解压路径。

```
PATH :=
"${PATH}:/usr/local/sbin:/usr/local/bin:/sbin:/bin:/usr/sbin:/usr/bin:${TOPDIR}/
toolchains/aarch64-linux-musleabi_for_x86_64-pc-linux-
gnu/bin:${TOPDIR}/toolchains/aarch64-linux-musleabi_for_x86_64-pc-linux-
gnu/aarch64-linux-musleabi/bin"
```

五、工具链配方 smart-gcc 定义 (recipes-toolchain/toolchain/smart-gcc_0.1.bb)

该配方的内容：根据指定的平台架构 (在 poky/build/conf/local.conf 里面定义的，可根据实际需求自行修改，如：MACHINE ??= "qemuarm64") 下载对应的toolchain压缩包，然后解压到 build/toolchains目录下。

每个bb文件可以定义多个task。
task的名称必须以"do_"作为前缀。
task函数的实现支持python和shell两种方式。

几点说明：

1. 由于下载的是RT-Thread官方最新的toolchain压缩包，而该压缩包可能会随时更新，所以就关闭了针对toolchain压缩包的MD5校验。

```
BB_STRICT_CHECKSUM = "0"
```

2. 目前提供了aarch64和riscv64两个架构的toolchain资源，后续可以继续增加扩展。
3. 主要实现了下载与解压操作。

六、文件系统配方 busybox 定义 (recipes-rootfs/rootfs/busybox_0.1.bb)

该配方的内容：从指定地址下载busybox源码包，然后解压，打patch，编译，并将生成 ext4.img 文件系统镜像安装到指定位置备用。

几点说明：

1. 编译busybox需要依赖工具链，通过下面设置可以提前触发工具链安装Task。

```
do_build_rootfs[depends] = "smart-gcc:do_install_toolchain"
```

2. 必须提供源码资源的md5校验码
3. 编译busybox需要一些依赖文件，如头文件和基础库之类。
4. 通过 mke2fs 工具无需root就能生成ext4.img，不可使用"sudo mount"这种方式，以及任何需要sudo操作的指令。因为bitbake运行在一个fakeroot环境，不支持sudo操作。

七、内核配方 rt-smart 定义 (recipes-kernel/rt-thread/rt-smart_0.1.bb)

该配方的内容：从指定的地址下载rt-thread源代码，以及编译过程中依赖的其它源代码资源，编译生成rt-smart kernel文件，然后安装到指定位置备用。

几点说明：

1. 本配方中的所有资源都是git资源，需要指定branch, protocol, name, subdir等信息。
 - branch, 指定分支；
 - protocol, 指定协议类型；
 - name, 由于指定了多个资源，因此可以设置不同的名称以便后面设置各自仓库的哈希值；
 - subdir, 用于将资源存储到不同的位置，避免覆盖；
2. 可以通过"SRCREV_\$name"设置对应仓库的哈希值，也可以直接设置为"AUTOINC"表示获取最新代码。
3. 由于在编译rt-thread的时候，需要通过"scons --menuconfig"来生成"~/env"环境，但是Bitbake方式不支持menuconfig操作，所以通过提前将env, packages, sdk三个依赖库下载，然后手动构建env方式完成
4. 正常在编译rt-thread的时候，需要通过"pkgs --update"来安装lwext4 package，但是Bitbake方式执行该操作容易发生错误，所以通过提前将lwext4仓库下载，以便与kernel一起编译。
5. 定义了 do_build_kernel 编译 rt-smart 内核。当然了，它也需要依赖toolchain的安装。

6. 定义了 do_build_all 用于同时编译 busybox 和 rt-smart, 以便将ext4.img和kernel一起生成出来。
7. 分别对三个清理任务: do_clean, do_cleansstate, do_cleanall 进行了扩展, 不仅清理内核配方自身的, 同时清理文件系统配方和工具链配方的。再重新构建之前需要先clean一下。

八、为什么每个配方都使用了自定义的Task？

构建指定recipe的完整的任务链为：

```
do_fetch → do_unpack → do_patch → do_configure → do_compile → do_install → do_build
```

这些都是内置的task, 然后还会依赖一堆隐藏的task, 导致首次构建过程中需要下载缓存数百个软件包。

通过自定义task, 然后指定自定义的task进行构建, 则可以避开下载大量软件包问题。

注意：自定义task名称最好不要使用系统内置的task名称, 否则由于依赖关系清理不了在首次构建时依然会下载大量其它软件包。

可以通过如下命令查看指定recipe的任务列表：

```
$ bitbake -c listtasks $recipe-name
```

可以通过如下命令查看指定recipe的各个任务的依赖关系 (结果输出到文件 task-depends.dot, 文本方式打开即可)：

```
$ bitbake -g $recipe-name
```

九、目前支持的常用操作指令

1. 安装toolchain:

```
$ bitbake smart-gcc -c install_toolchain
$ bitbake smart-gcc -c clean
```

2. 编译busybox并生成ext4.img (会先判断toolchain是否安装, 否则先安装)

```
$ bitbake busybox -c build_rootfs
$ bitbake busybox -c clean
```

3. 编译rt-smart内核 (会先判断toolchain是否安装, 否则先安装)

```
$ bitbake rt-smart -c build_kernel
```

4. 编译busybox及rt-smart内核

```
$ bitbake rt-smart -c build_all
```

5. 同时清除toolchain, busybox, rt-smart的编译数据

```
$ bitbake rt-smart -c clean/cleansstate/cleanall
```

- clean - 清除掉tmp下的编译目录；
- cleansstate - 清除掉编译状态；
- cleanall - 同时会清除掉下载的源文件（慎用，否则又得下半天）；

十、如何增加新的bsp支持

目前支持两种bsp的编译，可以通过修改poky/build/conf/local.conf里面的MACHINE变量进行切换。

```
# MACHINE ??= "qemuarm64" #编译aarch64  
MACHINE ??= "qemuriscv64" #编译riscv64
```

如果想支持更多的bsp编译，需要修改的几个地方：

1. 增加Toolchain支持：修改recipe_toolchain/toolchain/smart-gcc.bb, 增加新的toolchain的下载信息。
2. 增加Busybox编译支持：参照01_qemuarm64.diff, 增加新的patch，名称为：01_\$MACHINE.diff，该patch文件主要是定义编译工具及参数。注意需要同时提供依赖的header和lib文件。
3. 增加bsp编译支持：目前的rt-smart.bb支持aarch64和riscv64的编译，可以在里面增加新的bsp支持；如果新的bsp编译逻辑差异性很大，可以新增bb文件。
4. 增加qemu启动脚本支持：如果需要qemu启动，可参考tools/run_qemuarm64.sh增加一个新的run_\$MACHINE.sh启动脚本。
5. 添加新的Toolchain PATH：修改conf/layer.conf里面PATH定义，将新的Toolchain PATH添加进去。